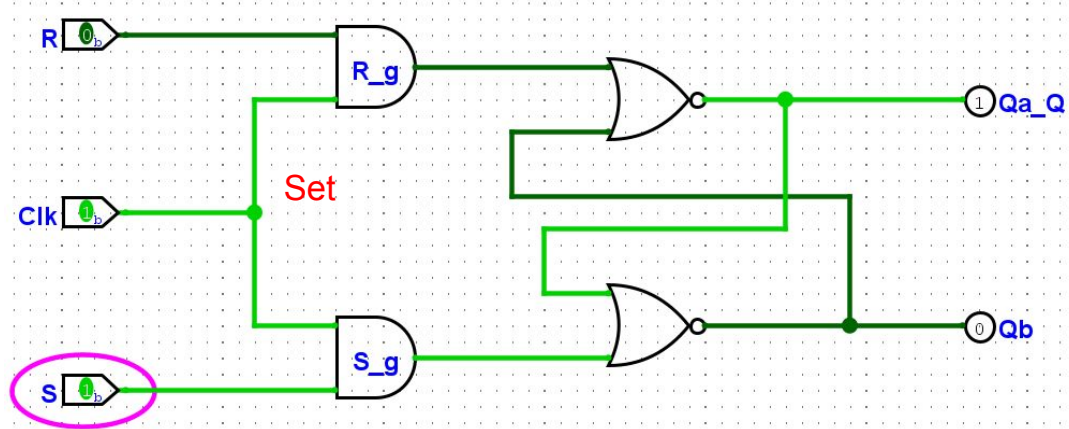
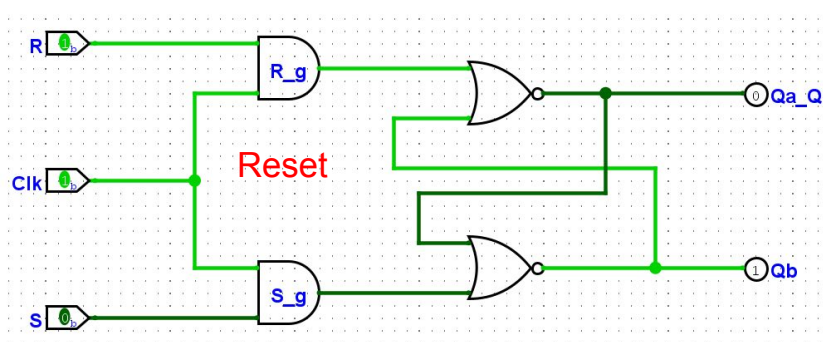


Lab 3

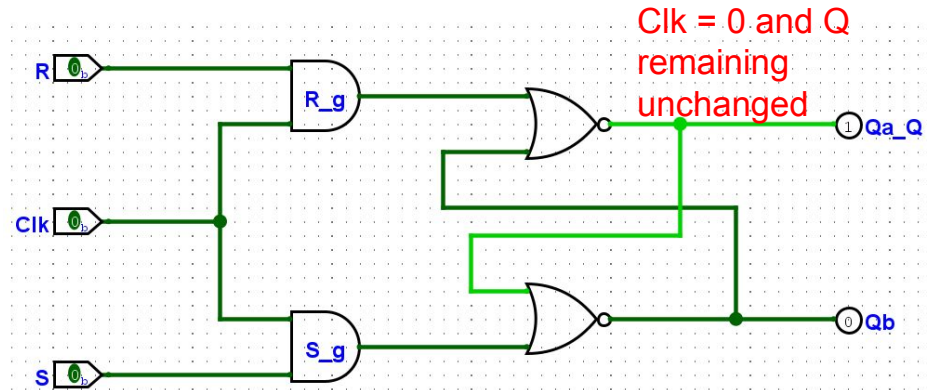
Part 1

Gated RS Latch-Set and Reset Verification

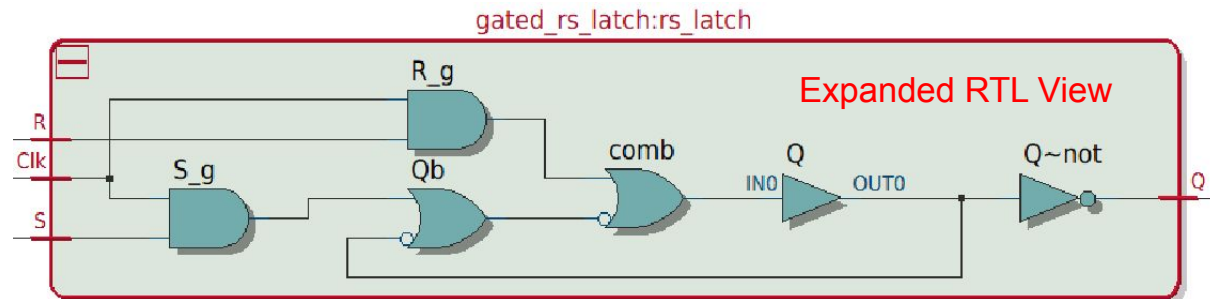


R	Clk	S	Qa_Q	Qb
0	0	0		
0	0	1		
0	1	0		
0	1	1	1	0
1	0	0		
1	0	1		
1	1	0	0	1
1	1	1	0	0

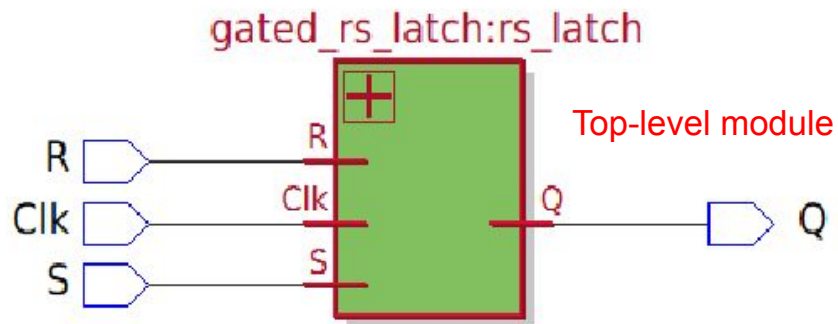
Red means Hold previous state



Gated RS Latch: Verilog Implementation and RTL Structure



The top-level module instantiates a reusable gated RS latch built from two AND gates and two cross-coupled NOR gates.



```
module gated_rs_latch(  
    input Clk,  
    input R,  
    input S,  
    output Q  
);  
  
    wire R_g, S_g, Qa, Qb;  
  
    and (R_g, R, Clk);  
    and (S_g, S, Clk);  
  
    nor (Qa, R_g, Qb);  
    nor (Qb, S_g, Qa);  
  
    assign Q = Qa;  
  
endmodule
```

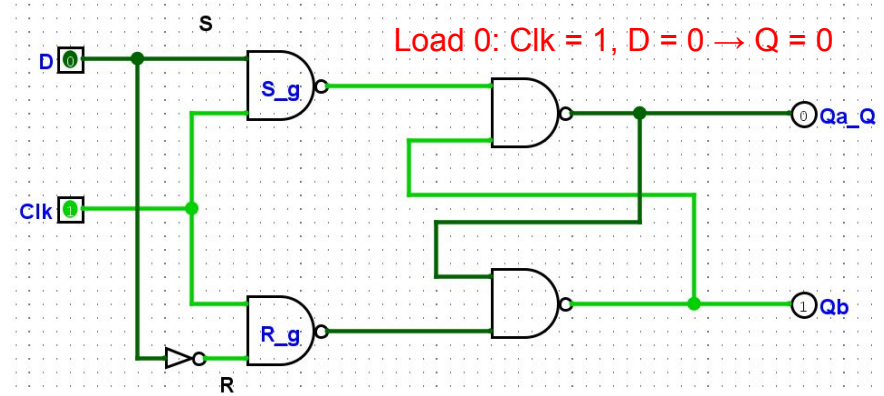
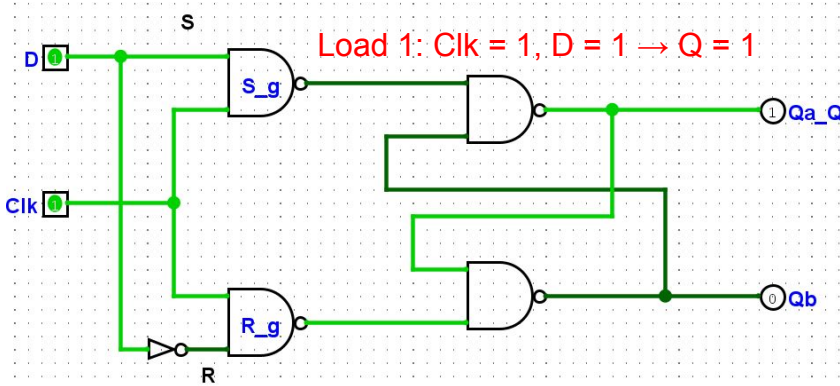
Gated RS Latch

```
module main(  
    input Clk,  
    input R,  
    input S,  
    output Q  
);  
  
    //Part I start:  
    gated_rs_latch rs_latch(  
        .Clk(Clk),  
        .R(R),  
        .S(S),  
        .Q(Q)  
    );  
    //Part I end  
endmodule
```

main.v

Part 2

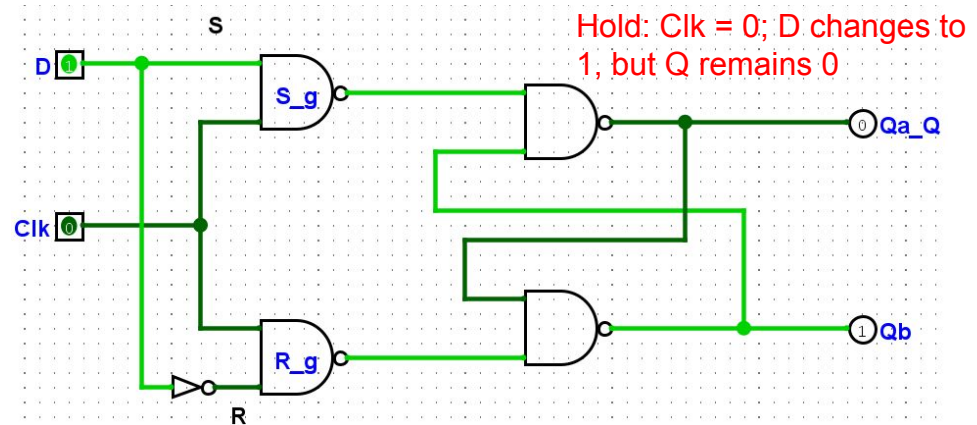
Gated D Latch: Load and Hold Verification



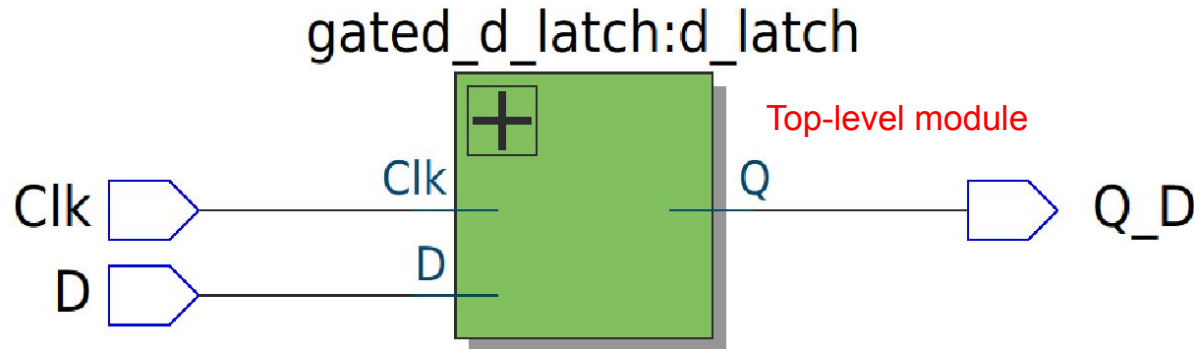
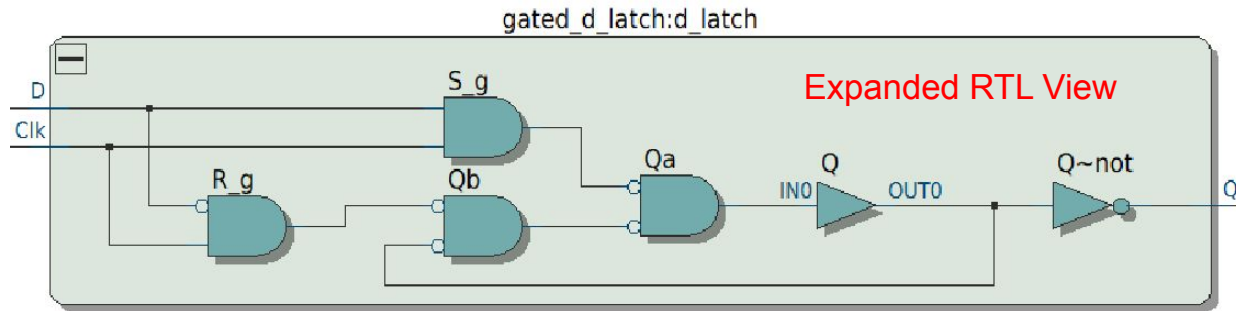
Gated D Latch State Table

D	Clk	Qa_Q	Qb
0	0		
0	1	0	1
1	0		
1	1	1	0

Clk = 0: outputs retain their previous state



Gated D Latch: Load and Hold Verification



```

module gated_d_latch(
    input D,
    input Clk,
    output Q
);

    wire R, S_g, R_g, Qa, Qb;

    assign R = ~D;

    assign S_g = ~(D & Clk);
    assign R_g = ~(R & Clk);

    assign Qa = ~(S_g & Qb);
    assign Qb = ~(R_g & Qa);

    assign Q = Qa;

endmodule
    
```

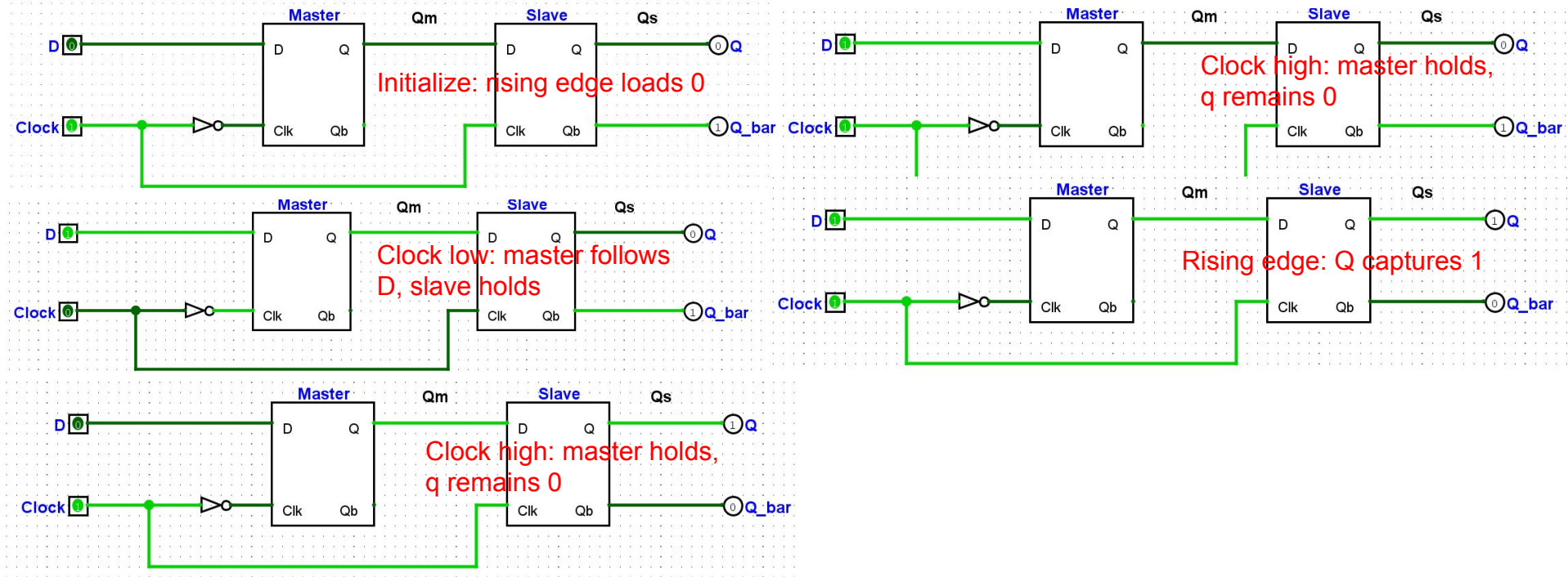
```

//Part II start:
    gated_d_latch d_latch(
        .D(D),
        .Clk(Clk),
        .Q(Q_D)
    );
//Part II end
    
```

main.v

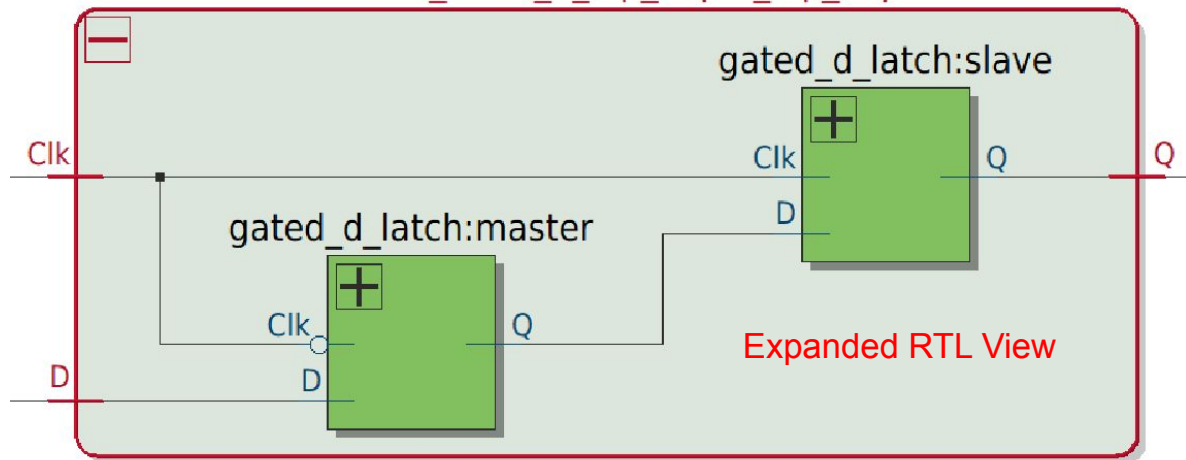
Part 3

Master-Slave D Flip-Flop: Rising-Edge Verification



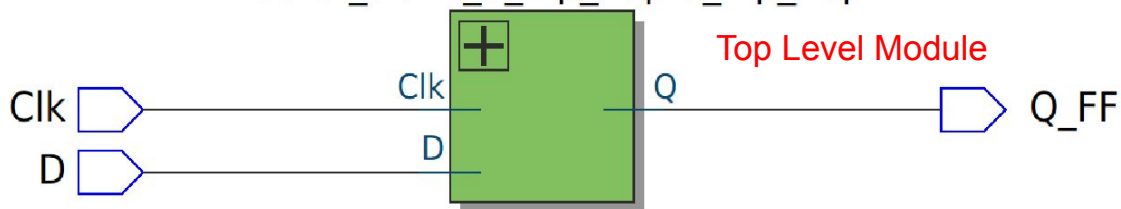
Master-Slave D Flip-Flop: Verilog Implementation and RTL Structure

master_slave_d_flip_flop:d_flip_flop



Expanded RTL View

master_slave_d_flip_flop:d_flip_flop



Top Level Module

```

module master_slave_d_flip_flop(
    input D,
    input Clk,
    output Q
);

    wire Clk_not, Qm;

    assign Clk_not = ~Clk;

    gated_d_latch master(
        .D(D),
        .Clk(Clk_not),
        .Q(Qm)
    );

    gated_d_latch slave(
        .D(Qm),
        .Clk(Clk),
        .Q(Q)
    );

endmodule
    
```

Master Slave
D Flip Flop

```

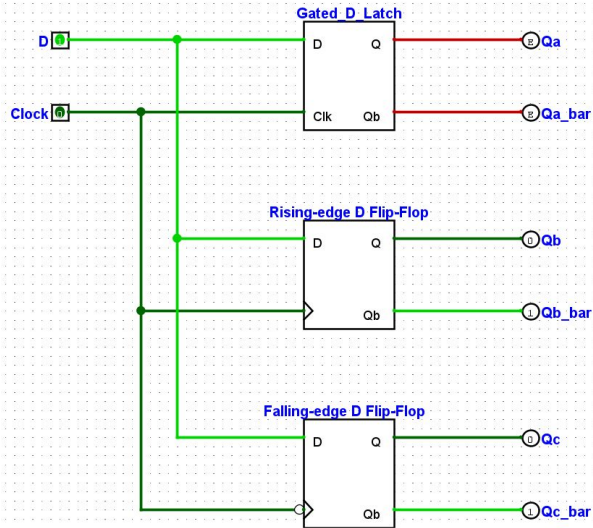
//Part III start:
master_slave_d_flip_flop d_flip_flop(
    .D(D),
    .Clk(Clk),
    .Q(Q_FF)
);
//Part III end
    
```

main.v

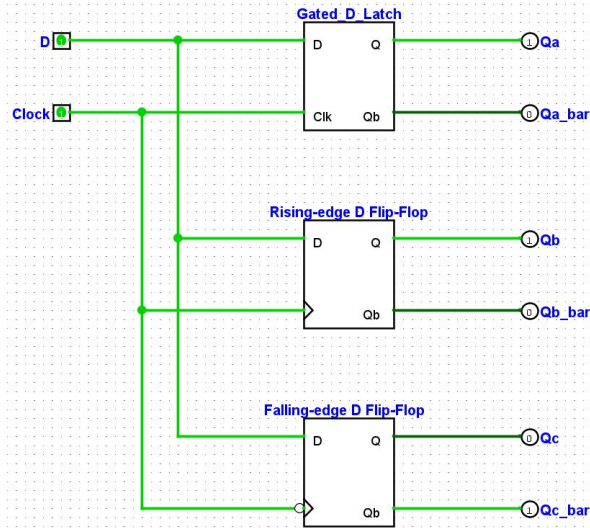
Part 4

D Latch and D Flip-Flops: Level and Edge Response

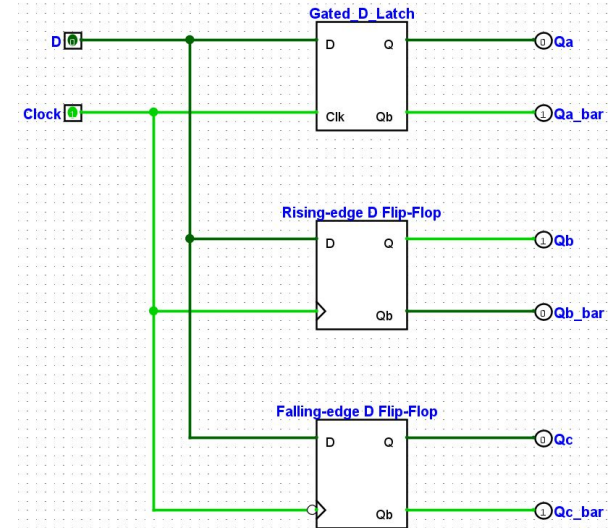
Clock low: D changes to 1; all outputs hold 0.



Rising edge: Qa & Qb become 1; Qc holds 0

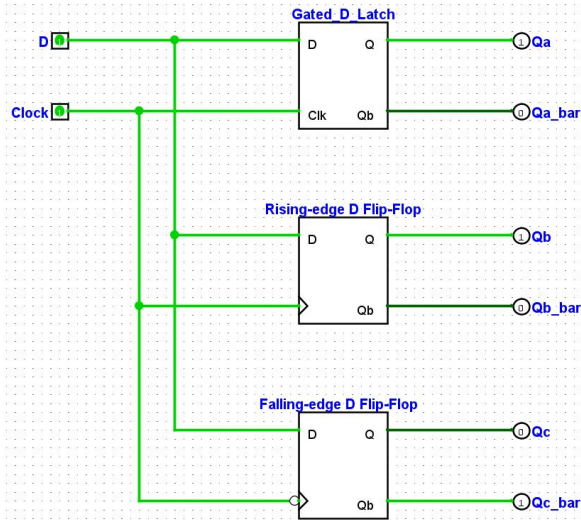


Clock high: D changes to 0. Qa follows, while Qb & Qc hold

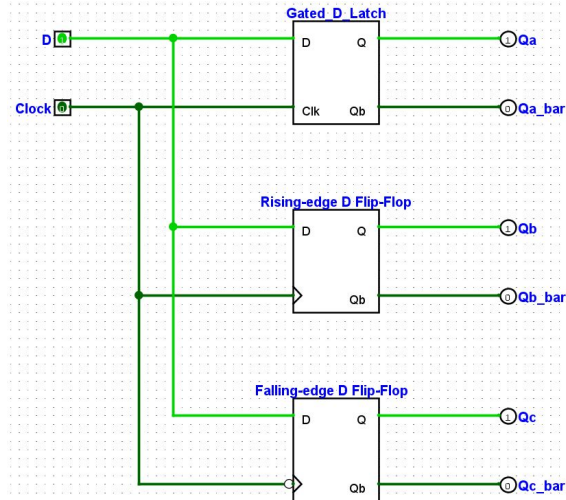


D Latch and D Flip-Flops: Level and Edge Response cont.

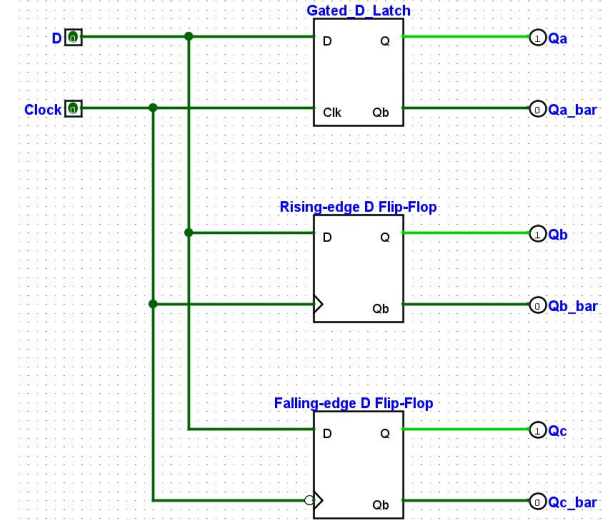
Clock high: D changes to 1. Qa follows, while Qb & Qc hold



Falling edge: Qa & Qb remain 1



Clock low: D changes to 0. All outputs hold 1



D Latch and D Flip-Flops: Behavioral Verilog Implementation

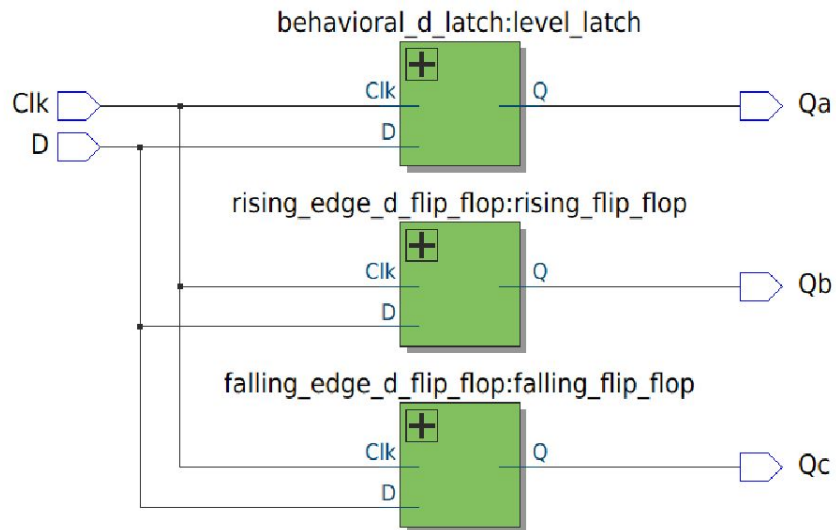
```
module behavioral_d_latch(  
    input D,  
    input Clk, D Latch  
    output reg Q  
);  
  
    always @(D or Clk) begin  
        if (Clk)  
            Q <= D;  
    end  
  
endmodule
```

```
//Part IV start:  
behavioral_d_latch level_latch(  
    .D(D),  
    .Clk(Clk),  
    .Q(Qa) main.v  
);  
  
rising_edge_d_flip_flop rising_flip_flop(  
    .D(D),  
    .Clk(Clk),  
    .Q(Qb)  
);  
  
falling_edge_d_flip_flop falling_flip_flop(  
    .D(D),  
    .Clk(Clk),  
    .Q(Qc)  
);  
//Part IV end
```

```
module rising_edge_d_flip_flop(  
    input D,  
    input Clk, Rising Edge  
    output reg Q  
);  
  
    always @(posedge Clk) begin  
        Q <= D;  
    end  
  
endmodule  
  
module falling_edge_d_flip_flop(  
    input D,  
    input Clk, Falling Edge  
    output reg Q  
);  
  
    always @(negedge Clk) begin  
        Q <= D;  
    end  
  
endmodule
```

D Latch and D Flip-Flops: RTL Structure

Top Level Module



Expanded RTL View

